// Prime Numbers

Agenda:

1. Check if given no. is prime
2. find all primes from 1 to N (sieve Algo.)
3. find count of divisors of all numbers from 1 to N.
4. find smallest Prime factor (spf) of all no. from 1 to N.

① Checking if a Number is Prime

↳ prime → has only 2 factors → 1 and itself

- instead of checking upto N check upto $\sqrt{N}$ because factors always appear in pairs.

⇒ $\boxed{\text{Logic}}$: for every i where $i \times i \le N$, if i divide N, count i and N/i as factors.

⇒ $\boxed{\text{T.C}}$: $O(\sqrt{N})$.

⇒ $\boxed{\text{S.C}}$: $O(1)$.

Code →

```
public boolean isPrime (int n) {
    if (n <=1) return false;
    int count =0;
    for (int i=1 ; i*i <=n ; i++){
        if (n%i ==0){
            if (i*i ==n) { count++ } ;      // perfect sq., count once
            else count +=2;                 // Count both i & n/i
        }
    }
    return count == 2;
}
```

② find all primes between 1 to N (Sieve of Eratosthenes)

- checking each individually will take $N\sqrt{N}$

optimised → mark multiples of primes as non-primes

T.C → $O(N \log(\log N))$

S.C → $O(N)$ → to store the boolean array

T.C of sieve

i	j	# iterations
2	4, 6, 8..	~ N/2
3	9, 12, 15..	~ N/3
4	—	—
5	25, 30..	~ N/5

$\Rightarrow N\left(\frac{1}{2}+\frac{1}{3}+\frac{1}{5}+\frac{1}{7}\dots\right)$

$\le \left(\frac{1}{2}+\frac{1}{3}+\frac{1}{4}+\frac{1}{6}\dots\right)$

$\int \frac{1}{x}\,dx = \log x$

approx. $\boxed{T.C < N \log N}$

Code →

```
public boolean [ ] sieve (int n){
    boolean[] isPrime = new boolean [n+1];
    for (int i=2; i<=n ; i++) isPrime [i] = true;
    for ( int i=2 ; i*i <=n ; i++){
        if (isPrime[i] ){
            for(int j = i*i ; j ≤ n ; j +=i ){
                isPrime[j] = false;
            }
        }
    }
    return isPrime;
}
```

---

(3) Count of Divisors for All Numbers (1 to N).

Logic: Use a techique similar to sieve. For every no. i, increment the count of all of its multiples. ~~Public~~ Every time you hit a mutiple j, it means i is a factor of j.

T.C: $O(N \log N)$

S.C: $O(N)$

Code →

```
public int [ ] countDivisors(int n ){
    int[] counts = new int [n+1];
    for (int i=1 ; i<=n ; i++){
        for(int j = i ; j <=n ; j +=i ){
            counts [j]++ ;        //i is a factor of j
        }
    }
    return counts;
}
```

---

(4) Smallest Prime Factor (SPF)

↳ smallest prime that divides a number i

Logic: Modify the sieve. Instead of marking T/F, if a no. j doesn't have an SPF yet, set SPF[j] = i.

T.C = $O(N \log(\log N))$

Code → Public int[] getSPF (int n) {

```
int[] spf = new int[n+1];
for (int i=2 ; i<=n ; i++){spf[i] = i};
```

↳ initialise with itself

```
for (int i = 2 ; i*i <= n ; i++){
    if (spf[i] == i){        // is prime
        for (int j = i*i ; j <= n ; j += i){
            if (spf[j] == j) spf[j] = i ;
                // Mark only if not already marked
        }
    }
}
return spf;
}
```

---

(5) # Fast Factorization using SPF

Find ↳ total no. of divisors for a number N using its prime exponent.

if $N = P_1^a * P_2^b$, no. of divisors is $(a+1)(b+1)$.

Logic : Repeatedly divide N by its SPF [N] to count the exponent of each prime factor.

T.C : $O(\log N)$.

Code → public int countDivisorsfast (int n, int[] SPF) {

```
int totalDivisors = 1
while (n>1) {
    int p = SPF[n] ;
    int count = 0;
    while (n % p == 0){
        count++ ;
        n /= p;
    }
    totalDivisors *= (count+1)        // (a+1)(b+1)
}
return totalDivisors;
}
```